Linux perf Based Memory Access Pattern Analysis

Pointer chasing, STREAM-lite, stride locality, row/column traversal, and tiled matmul

Undergraduate systems project | Linux, C, Python, perf automation

Motivation and Questions

Goal: observe software-visible memory bottlenecks through small reproducible benchmarks.

Question 1: how do runtime and locality change as the working set grows?

Question 2: which patterns behave as latency-bound versus bandwidth-bound?

Question 3: what simple software changes improve reuse most clearly?

Key constraint: Hardware PMU events were unsupported, so final interpretation relies mainly on runtim

Benchmark Suite and Workflow

Benchmarks: pointer_chase, stream_lite, stride_access, row_col_traversal, optional matmul_tiled.

Each benchmark emits one CSV line with problem size, timing, and one benchmark-specific metric.

Automation: collect_perf.sh -> parse_perf.py -> plot_results.py.

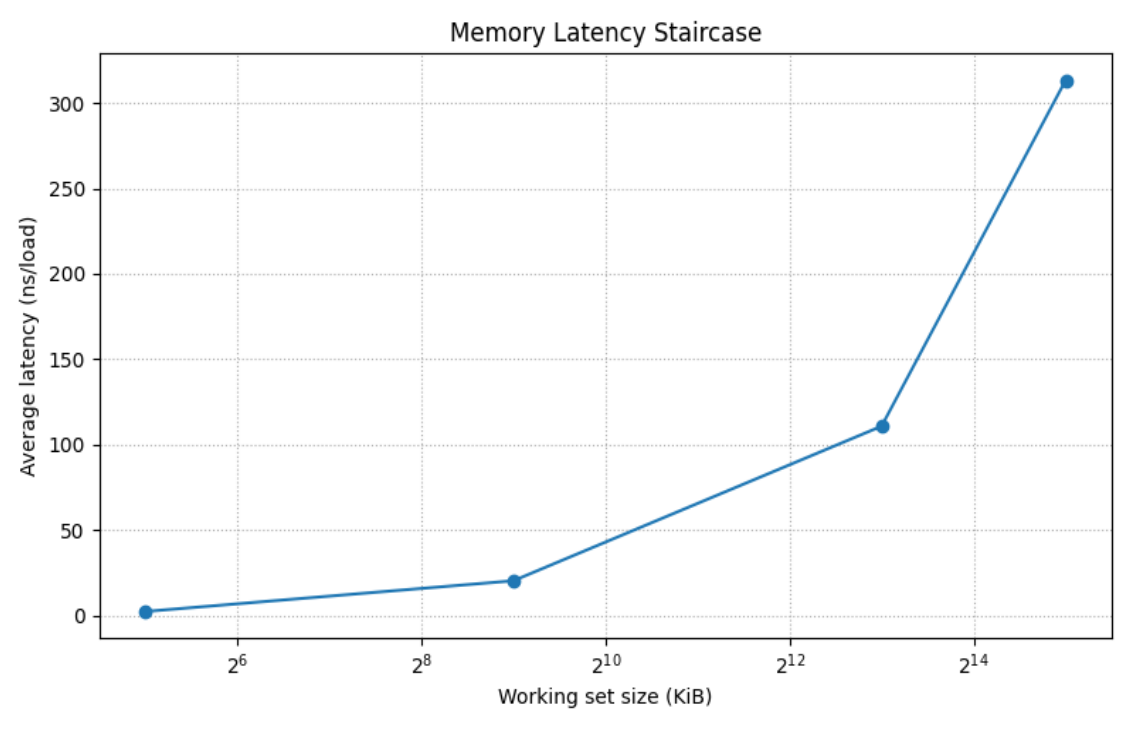
Controls: core pinning, warmup, repeated runs, fixed compiler flags.

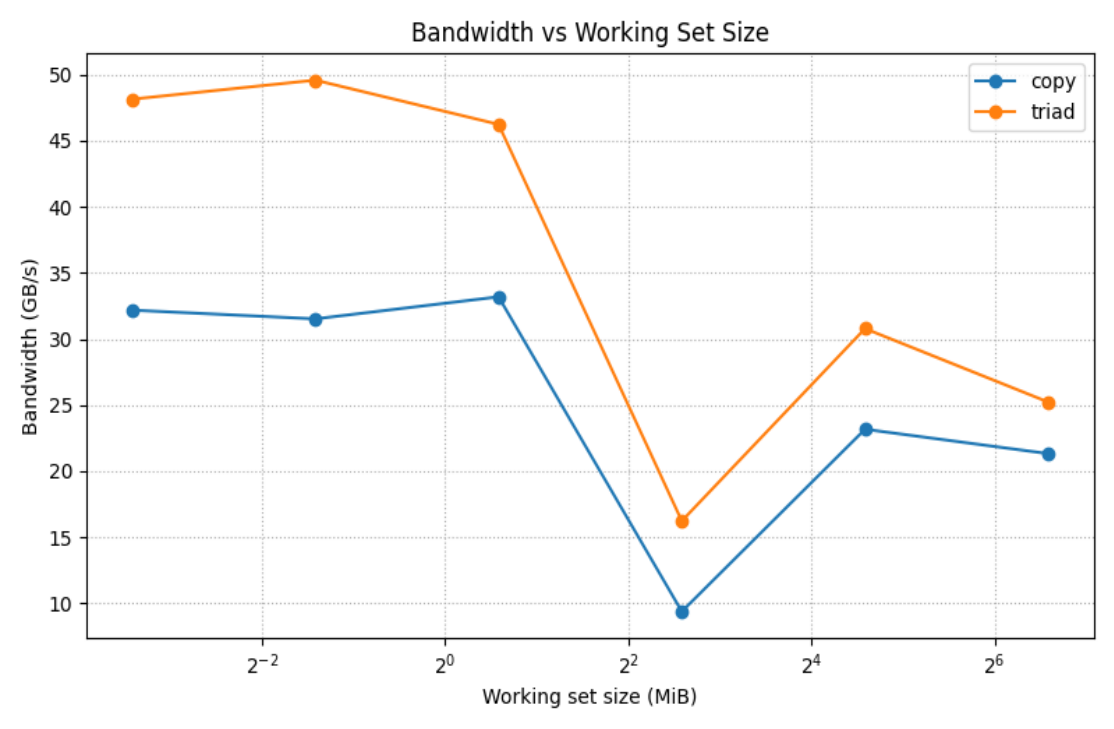
Preferred events: cycles, instructions, cache-misses, LLC, L1D, dTLB.

Actual usable events on this host: task-clock, cpu-clock, page-faults, context-switches, cpu-migrations.

Result: timing trends are strong, but cache-level attribution must stay conservative.

Strongest Results: Latency Staircase and Streaming Bandwidth



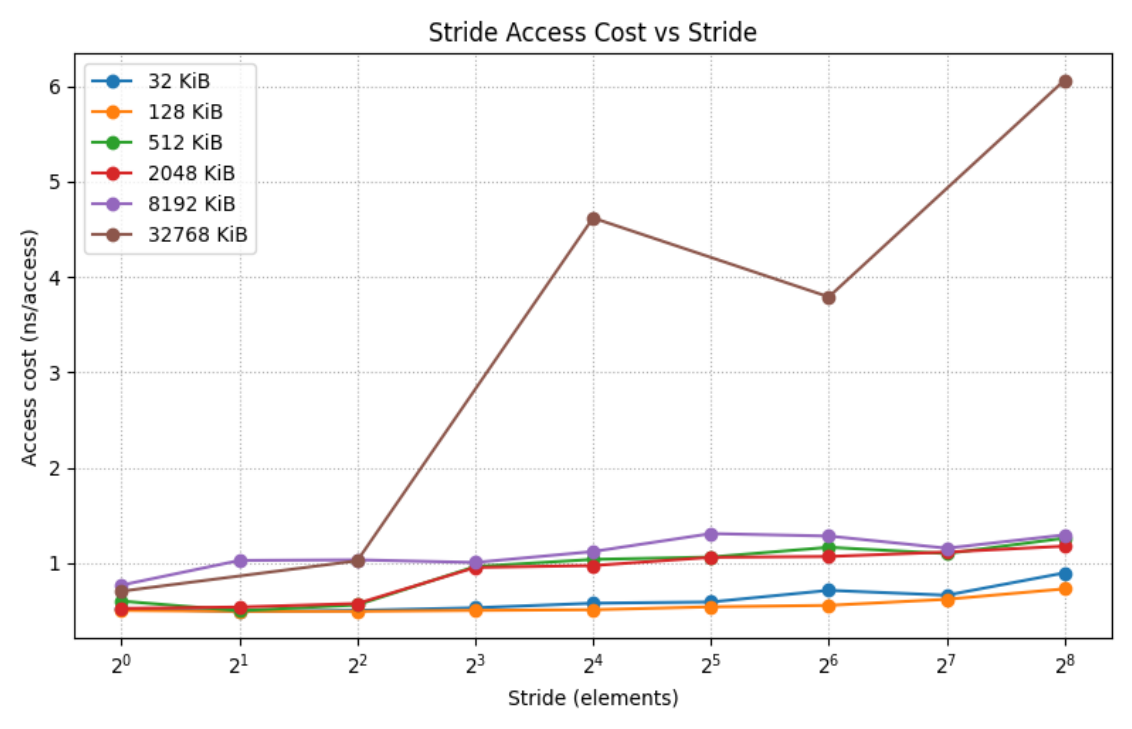


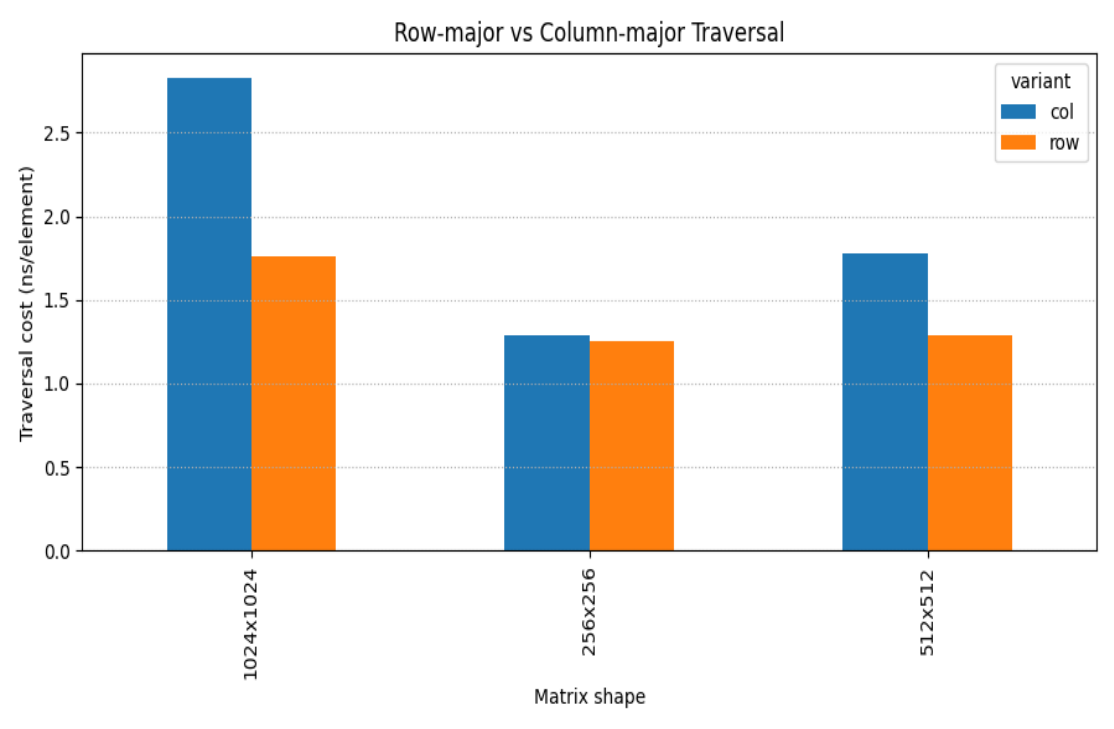
Pointer chase rose from about 20.2 ns/load at 512 KiB to 110.8 ns/load at 8 MiB and 313.4 ns/load at 32 MiB.

STREAM-lite sustained about 32-33 GB/s for copy and 46-49 GB/s for triad on small to medium working sets.

Interpretation: irregular dependent access is latency-bound, while regular streams expose far more overlap.

Moderate Results: Stride and Traversal Order



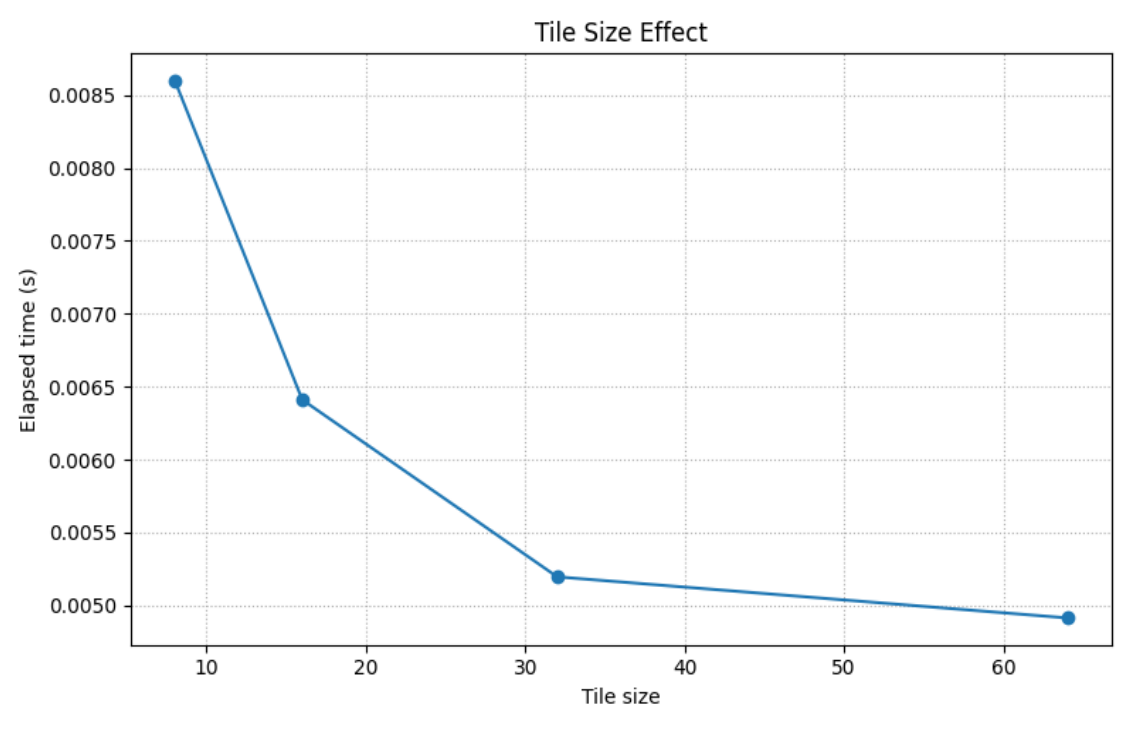


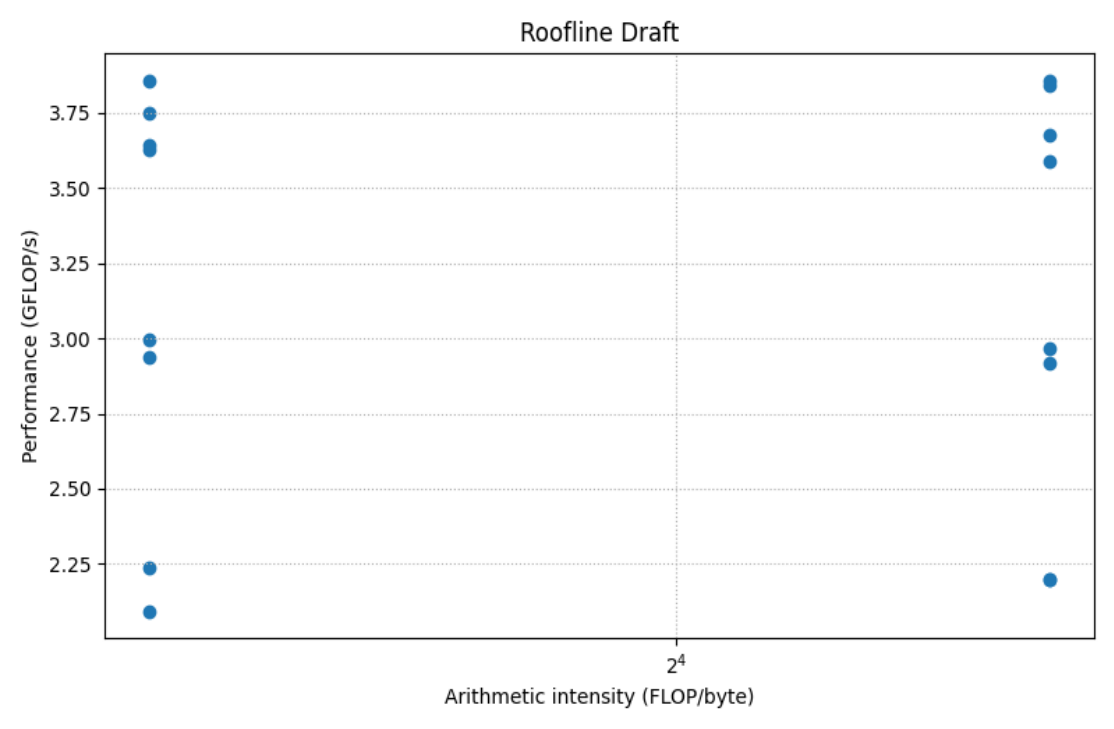
Stride effect was small for tiny footprints, but stronger for large ones: at 32 MiB, stride 256 reached about 6.06 ns/access.

Row-major stayed faster than column-major, especially at 1024x1024: about 1.76 ns/element versus 2.83 ns/element.

These trends match locality expectations, but the effect sizes are weaker than pointer chasing on this host.

Best Optimization Example: Tiled Matrix Multiply





For 128x128, throughput improved from about 2.17 GFLOP/s at tile 8 to 3.80 GFLOP/s at tile 64.

For 256x256, throughput improved from about 2.20 GFLOP/s at tile 8 to 3.85 GFLOP/s at tile 64.

Takeaway: explicit reuse through blocking produced the clearest software-level gain in the project.

Takeaways, Limitations, and Next Step

Clear results: pointer-chasing latency staircase, streaming bandwidth plateau, and tiled-matmul reuse benefit.

Weaker but consistent trends: wider strides and column-major traversal generally cost more.

Main limitation: hardware PMU events were unavailable, so cache-level attribution remains indirect.

Next step: rerun the exact same suite on a Linux host with working PMU support for cycles, instructions, and cache-miss counters.